

CoursePilot: Proposed Repository Structure

Backend Skeleton & Initial Setup

Proposed Folder Structure

```
coursepilot/
├── .gitignore                # Python + Node + Firebase + IDE exclusions
├── README.md                 # Project overview and quick start
├── frontend/                 # Frontend team's workspace
│   └── .gitkeep              # Placeholder (frontend team adds Next.js)
├── backend/                  # Backend FastAPI application
│   ├── app/                  # Makes app a Python package
│   │   ├── __init__.py       # FastAPI entry point
│   │   ├── main.py           # Environment variables & settings
│   │   ├── config.py         # Auth middleware & shared dependencies
│   │   └── dependencies.py
│   ├── models/               # Pydantic models (data validation)
│   │   ├── __init__.py       # Request/response types from ERD
│   │   └── schemas.py
│   ├── routes/               # API endpoints (organized by feature)
│   │   └── __init__.py
│   ├── tasks/                # Celery background tasks
│   │   └── __init__.py
│   └── utils/                # Shared utility functions
│       └── __init__.py
├── celery_app.py             # Celery configuration
├── requirements.txt           # Python dependencies
├── .env.example              # Template for environment variables
├── README.md                 # Backend setup instructions
├── docs/                     # Shared documentation
│   └── .gitkeep              # Placeholder for docs
```

Key Files & Their Purpose

File	Purpose
backend/app/main.py	FastAPI application with CORS, health check endpoint, and router imports
backend/app/config.py	Loads settings from .env (Firebase credentials, Redis URL, etc.)
backend/app/dependencies.py	Firebase Auth middleware with <code>get_current_user()</code> dependency
backend/celery_app.py	Celery configuration with Redis broker for background tasks
backend/requirements.txt	All Python dependencies (FastAPI, Firebase, Celery, etc.)
backend/.env.example	Template showing required environment variables
backend/README.md	Detailed setup instructions for backend
.gitignore	Prevents committing <code>venv/</code> , <code>.env</code> , <code>__pycache__</code> , <code>node_modules/</code> , etc.
README.md	Project overview, tech stack, and quick start for whole team
frontend/.gitkeep	Empty placeholder so Git tracks the frontend folder

Branch Strategy

Proposed branching workflow:

- `main` → Production-ready code (protected, only merge from `dev`)
- `dev` → Integration branch where all features merge
- `feature/*` → Individual work branches (e.g., `feature/syllabus-upload`)

Workflow: Create feature branch from `dev` → make changes → PR to `dev` → merge → delete feature branch

Python Dependencies (requirements.txt)

- `fastapi==0.115.0` - Web framework
- `uvicorn[standard]==0.32.0` - ASGI server
- `python-dotenv==1.0.1` - Environment variable management
- `firebase-admin==6.5.0` - Firebase SDK (Auth, Firestore, Storage)
- `celery==5.4.0` - Distributed task queue
- `redis==5.2.0` - Message broker for Celery
- `python-multipart==0.0.9` - File upload support

What Will Be Ready After Setup

- Backend API running at <http://localhost:8000>
- Interactive API docs at <http://localhost:8000/docs> (Swagger UI)
- CORS configured for Next.js frontend (localhost:3000)
- Firebase Auth middleware ready (needs credentials in .env)
- Celery configuration ready (needs Redis running)
- Proper folder structure for adding routes and tasks
- Empty frontend/ folder for frontend team to populate

Quick Start (After Setup)

```
# Backend
cd backend
python -m venv venv
source venv/bin/activate # Windows: venv\Scripts\activate
pip install -r requirements.txt
uvicorn app.main:app --reload

# Visit http://localhost:8000/docs

# Celery (separate terminal, needs Redis running)
celery -A celery_app worker --loglevel=info
```

Notes

- All `__init__.py` files will be empty (just for Python package structure)
- Firebase credentials will NOT be in the repo (added to .gitignore)
- Everyone will need to create their own .env file based on .env.example
- This setup follows FastAPI best practices and is production-ready
- Frontend team can set up Next.js independently in the frontend/ folder